



2.16. Основы алгоритмов, часть 3

Привет! На связи шпаргалка к уроку 2.16. Основы алгоритмов, часть 3.

Сегодня мы рассмотрим одну из самых сложных для начинающих разработчиков тем, а именно рекурсию. На основании рекурсии мы научимся строить дерево, а также изучим продвинутые виды сортировок, которые не будет стыдно использовать в реальном проекте.



Так как в шпаргалке к уроку 2.7 уже было рассмотрено устройство Map, повторения в этой шпаргалке не будет.

Рекурсия

Плюсы рекурсии

Минусы рекурсии

StackOverflowError

Фибоначчи

Сортировки

Быстрая сортировка

Сортировка слиянием

Рекурсия

Рекурсия в программировании — задание алгоритма вычисления функции с использованием вызова ее самой.

Если объяснять простыми словами, то рекурсией называется ситуация, когда метод напрямую или через другие методы производит вызов самого себя.

В результате некорректного использования рекурсии возможна ситуация, когда метод будет до бесконечности вызывать сам себя, что без сомнений приведет к `StackOverflowError`. Эту ошибку мы воспроизведем в следующем блоке.

Зачастую рекурсивную задачу можно решить с помощью циклов, однако код получится несколько длиннее. Рекурсия и циклы имеют как свои плюсы, так и минусы. Рассмотрим их вкратце.

Плюсы рекурсии

1. Код получается короче и читабельнее, т. к. метод выполняет небольшую часть работы, а затем вызывает сам себя, повторно вычисляя изменившиеся условия.

Зачастую рекурсивные методы занимают не более 5 строк.

2. Код (за счет своей краткости) проще отлаживать.

Как мы упомянули ранее, код рекурсивного метода состоит из небольшого количества строк. Следовательно, нет никакой необходимости проверять каждый шаг, как в случае с циклами. Достаточно проверить работу одного шага, а также условие выхода из метода.

Минусы рекурсии

1. Затраты памяти.

Мы с вами уже изучали устройства памяти в Java, которые делятся на стек и кучу.

Вспомним стек. В нем хранятся некие сущности, которые называются `Frame` (фрейм). Эта сущность создается каждый раз, когда ваш код «заходит» в новый метод.

Следовательно, в случае рекурсии фреймов создается столько, сколько раз метод вызвал сам себя. Это, безусловно, накладывает дополнительные затраты памяти.

2. Затраты времени.

Исходя из прошлого пункта, помимо памяти замедляется и общее выполнение программы, т. к. создание фрейма - не тяжелая задача, но всё-таки времязатратная.

3. Шанс переполнить стек.

Некорректно написанный рекурсивный метод может вызвать переполнение стека.

Эта ошибка не обрабатывается через try-catch и может «вылететь» в любой момент, прервав ваше приложение.

4. Необходимость небольшой перестройки мышления.

Рекурсивные методы работают следующим образом: результат не будет возвращен окончательно до тех пор, пока вызовы не дойдут до «самого низа» и не сработает условие, когда вызовы дальше не имеют смысла.

Решим простую задачу.

Она стандартна для циклов и легко реализуется с их помощью. Коду необходимо начать с нуля и напечатать все числа (в пределах 100), которые кратны 10. Первым делом решим задачу с помощью цикла.

```
public static void printNumsWithCycle() {  
    for (int i = 0; i <= 100; i += 10) {  
        System.out.println(i);  
    }  
}
```

Теперь перепишем код с рекурсией.

```
public static void printNumsWithRecursion() {  
    printNumsWithRecursion(0);  
}  
  
public static void printNumsWithRecursion(int currentNum) {  
    System.out.println(currentNum);  
  
    if (currentNum < 100) {  
        printNumsWithRecursion(currentNum + 10);  
    }  
}
```

Оба способа выполняют один функционал, но даже на таком простом примере рекурсивная конструкция выглядит проще для восприятия.

StackOverflowError

Попробуем воспроизвести ошибку, которая возникает в случае некорректного написания рекурсивного метода. Ситуацию воспроизвести достаточно просто. В коде выше эта ситуация была покрыта за счет if, который в определенный момент заканчивал вызов метода, и стек вызовов постепенно закрывал лишние фреймы. Попробуем удалить if.

```
public static void printNumsWithRecursion(int currentNum) {  
    System.out.println(currentNum);  
    printNumsWithRecursion(currentNum + 10);  
}
```

Через некоторое время увидим следующий результат:

```
Exception in thread "main" 103760  
java.lang.StackOverflowError
```

А затем будет распечатан огромный стек вызовов, который привел к ошибке.

Фибоначчи

Перейдем к рассмотрению простых алгоритмов на базе рекурсии. Вы уже реализовывали метод по вычислению некоторого количества чисел последовательности в одном из домашних заданий на первом курсе. Напомним код:

```

public static void printFibonacci() {
    int a = 0;
    int b = 1;

    for (int i = 0; i < 5; i++) {
        System.out.print(a + " " + b + " ");
        a += b;
        b += a;
    }
    System.out.println();
}

```

Осуществим печать данной последовательности с помощью рекурсии.

В качестве условия выхода реализуем уже знакомый прием с if.

```

public static void printFibonacci(int num1, int num2, int currentStep, int count) {
    if (currentStep < count) {
        System.out.print(num1 + " ");
        printFibonacci(num2, num1 + num2, currentStep + 1, count);
    }
}

```

Код выглядит просто и опрятно.

Запустить его можно с помощью следующего вызова:

```

printFibonacci(0, 1, 0, 10);

```

Или можно написать запускающий метод, который и без нас передаст необходимые стандартные параметры:

```

public static void printFibonacci(int count) {
    int firstNumber = 0;
    int secondNumber = 1;
    int currentStep = 0;
    printFibonacci(firstNumber, secondNumber, currentStep, count);
    System.out.println();
}

```

Бонусом мы получили универсальный метод, который может построить любое количество чисел из последовательности. Ему достаточно для этого знать первые два числа и общее количество чисел.

Сортировки

В качестве реальных случаев применения рекурсии можно назвать две сортировки. Обе сортировки в среднем имеют сложность $O(n * \log_2 n)$, но есть особенности.

Быстрая сортировка в худшем случае приобретает сложность $O(n^2)$, но не требует дополнительной памяти.

Сортировка слиянием создает множество подмассивов, т. е. количество потребляемой памяти увеличивается.

Разберемся в коде.

Быстрая сортировка

Быстрая сортировка является одной из самых популярных рекурсивных сортировок. Она работает с одним массивом, т. е. не порождает дополнительных, которые занимают лишнюю память. В худшем случае (когда абсолютно все данные не отсортированы) такая сортировка имеет квадратичную сложность.

Ее использование опирается на то, что в большинстве ситуаций сортируемые данные являются частично отсортированными. Ситуации, когда абсолютно все данные отсортированы в обратном порядке, практически невозможны в реальных условиях. Если данные частично отсортированы, средняя сложность равна средней сложности сортировки слиянием, т. е. $O(n * \log n)$.

Логика данной сортировки подразумевает нахождение некоторого элемента (например, последнего) и использование его в качестве некоего разделителя. Далее происходит поиск пары элементов: первого элемента, что меньше разделителя, и первого элемента, что больше разделителя. Затем элементы меняются местами. И так до тех пор, пока все элементы не будут пройдены.

После череды взаимных свопов в левой части массива оказываются те элементы, что меньше разделителя, а в правой — те, что больше. Затем

разделитель устанавливается между двумя этими частями, и на каждой из половин рекурсивно запускается этот же алгоритм. Рассмотрим код:

```
public static void quickSort(int[] arr, int begin, int end) {
    if (begin < end) {
        int partitionIndex = partition(arr, begin, end);

        quickSort(arr, begin, partitionIndex - 1);
        quickSort(arr, partitionIndex + 1, end);
    }
}

private static int partition(int[] arr, int begin, int end) {
    int pivot = arr[end];
    int i = (begin - 1);

    for (int j = begin; j < end; j++) {
        if (arr[j] <= pivot) {
            i++;

            swapElements(arr, i, j);
        }
    }

    swapElements(arr, i + 1, end);
    return i + 1;
}
```

В данной сортировке используется уже знакомый по сортировкам метод swap:

```
private static void swapElements(int[] arr, int left, int right) {
    int temp = arr[left];
    arr[left] = arr[right];
    arr[right] = temp;
}
```

Разберем по шагам на примере сортировки массива {9, 4, 6, 5, 3, 5}.

1. Найдем разделитель. Например, последний элемент, т. е. 5.
2. Первым делом мы пройдем по всем элементам и перенесем элементы меньше разделителя в левую часть (поменяем местами с теми, что больше).

В нашем случае 9 больше 5, а 3 — меньше, следовательно, мы поменяем их местами.

Получим массив: {3, 4, 6, 5, 9, 5}.

3. Вставим разделитель в середину (между частями). Массив: {3, 4, 5, 5, 9, 6}.
4. Теперь на две обе половины вызовем шаг 1.
5. Левая часть будет состоять из 3 и 4. После прохождения всех шагов ничего не изменится, потому мы опустим этот момент.
6. Правая часть будет состоять из {5, 9, 6}.
7. В этой части за разделитель будет взята 6. 5 и 9 будут сравнены, а затем 6 поменяется местами с 9 (встанет в середину).
8. Получится массив {5, 6, 9}, который тоже будет разделен на два, но сортировать в нем нечего, потому опустим этот момент.

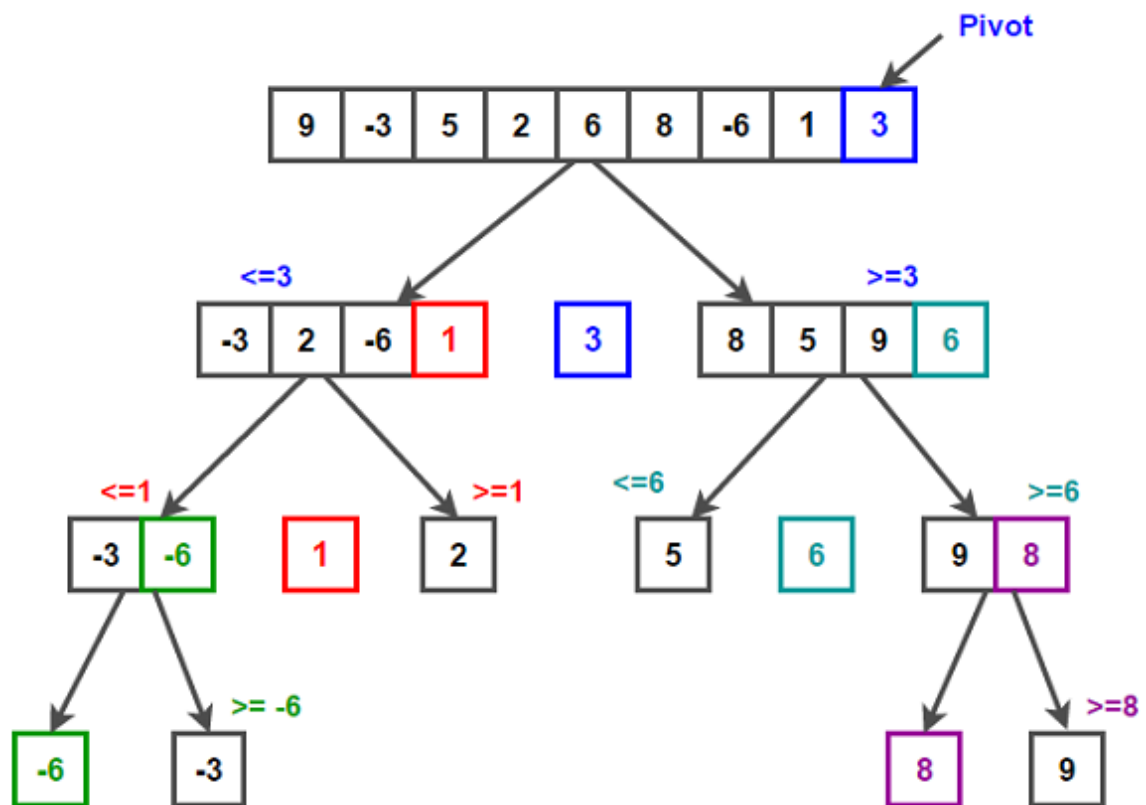
Стоит упомянуть, что позиция разделителя является достаточно дискуссионным вопросом.

Безусловно, удобнее всего будет выбрать центральный элемент в качестве разделителя, но его поиск может занять дополнительное время.

Потому (обычно) разделителем выбирают первый или последний элемент.

Реализации быстрой сортировки можно встретить с любым из этих вариантов.

▼ Схема



Сортировка слиянием

Данная сортировка осуществляется в две стадии.

На первой стадии массив разбивается пополам до тех пор, пока в его половинах не останется по одному элементу.

После этого происходит вторая стадия, которая на основе двух половин собирает один общий отсортированный массив.

Рассмотрим код первой стадии:

```
public static void mergeSort(int[] arr) {
    if (arr.length < 2) {
        return;
    }
    int mid = arr.length / 2;
    int[] left = new int[mid];
    int[] right = new int[arr.length - mid];

    for (int i = 0; i < left.length; i++) {
        left[i] = arr[i];
    }
}
```

```

    for (int i = 0; i < right.length; i++) {
        right[i] = arr[mid + i];
    }

    mergeSort(left);
    mergeSort(right);

    merge(arr, left, right);
}

```

Как видите, первая стадия происходит достаточно примитивно.

От одного большого массива создаются два новых, которые заполняются пополам.

Затем эти половинки тоже делятся пополам, и так до тех пор, пока `arr.length` не будет меньше двух. Если это происходит, значит, делить дальше нельзя.

В этот момент срабатывает вторая стадия — `merge`.

Код второй стадии:

```

public static void merge(int[] arr, int[] left, int[] right) {

    int mainP = 0;
    int leftP = 0;
    int rightP = 0;
    while (leftP < left.length && rightP < right.length) {
        if (left[leftP] <= right[rightP]) {
            arr[mainP++] = left[leftP++];
        } else {
            arr[mainP++] = right[rightP++];
        }
    }
    while (leftP < left.length) {
        arr[mainP++] = left[leftP++];
    }
    while (rightP < right.length) {
        arr[mainP++] = right[rightP++];
    }
}

```

В качестве параметров на вторую стадию приходит изначальный массив, а также его левая и правая половинки.

И на основе данных из двух половинок изначальный массив заполняется элементами.

Это происходит с помощью трех бегунков, которые независимо друг от друга двигаются по трем массивам и помещают текущий минимальный элемент из двух половинок на следующую позицию в основной массив.

Как вы должны были заметить, каждый вызов первого шага порождает два новых массива. Именно эти «накладные» расходы являются причиной отказа от сортировки слиянием в некоторых ситуациях.

▼ Схема

